Perceptron

Lê Thành Sách: LTSACH@hcmut.edu.vn

Dept. of Computer Science

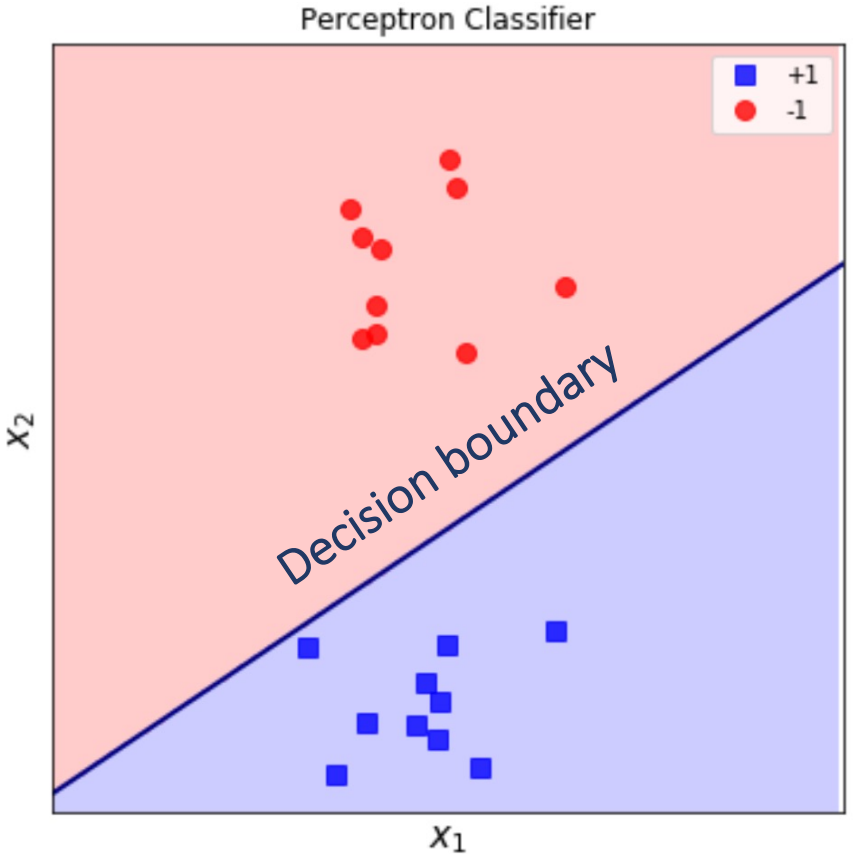
Ho Chi Minh City Univ. of Technology

Content

- What is?
- Computational Graph
- Model's prediction
- Learning
- Stochastic Gradient Descent (SGD)
- Perceptron Learning algorithm (PLA)
- PLA: Step-by-step
- Other kinds of classification models

What is?

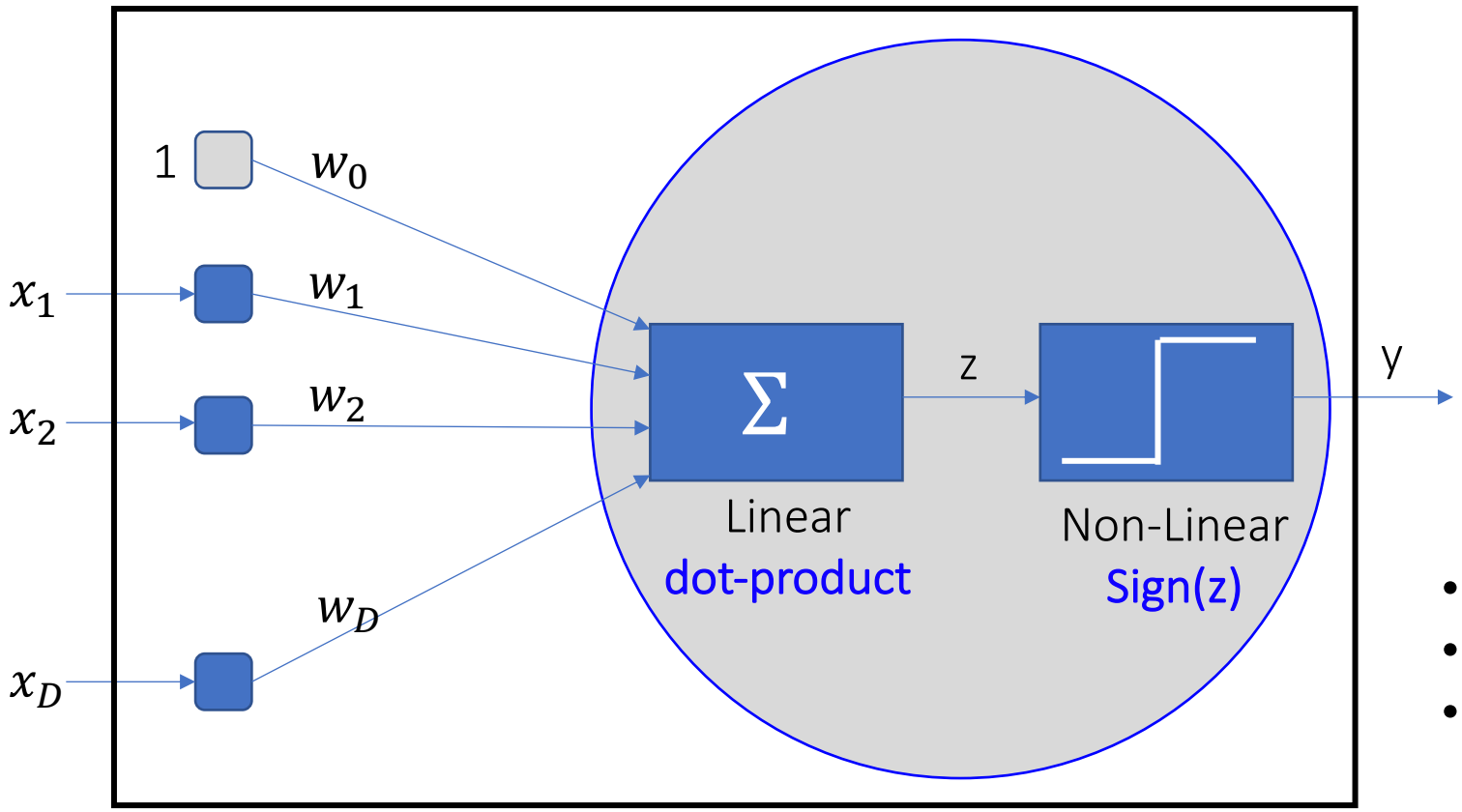
Perceptron: a computation model for solving 2-classes classification problems from linearly-separable datasets



Linearly Seperable: There exists a straight-line (2-D), a plane (3-D), or a hyper-plane (n-D) to sperate the feature space into two regions.

- one contains only the data points from +1
- and, the other contains only the data points from -1

Computational Graph

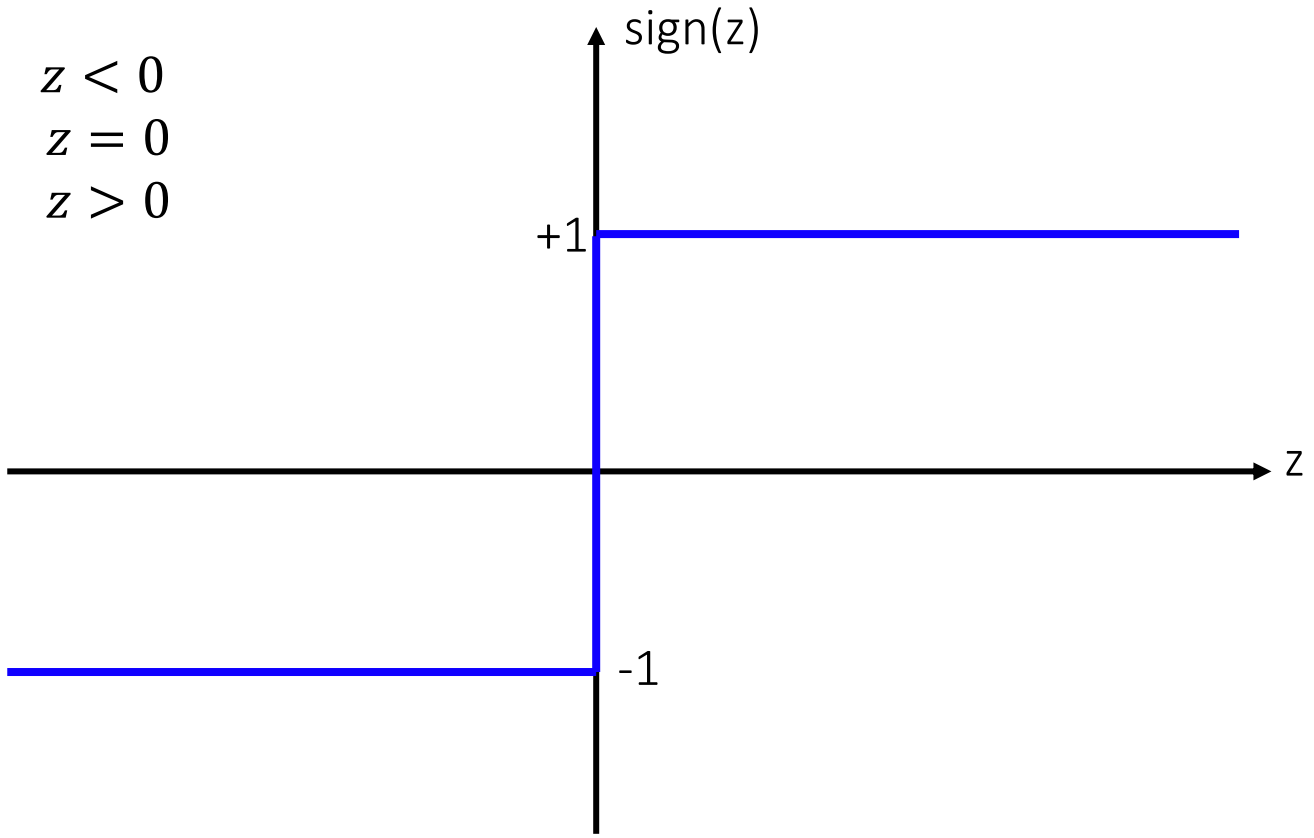


- Input: $x_1, x_2, \cdots, x_D$ (in D-dimension)
- Model's parameters: $w_0, w_1, w_2, \cdots, w_D$
- Output: y
  - Dom(y) = {-1, 0, +1}

Computational Graph

Sign function:

$$sign(z) = \begin{cases} -1 & z < 0 \\ 0 & z = 0 \\ +1 & z > 0 \end{cases}$$



Computational Graph

Model: $y = sign(w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_D x_D)$

IF:

$$\boldsymbol{w} = \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_D \end{bmatrix} = [w_0, w_1, \cdots, w_D]^T$$

$$\boldsymbol{x} = \begin{bmatrix} 1 \\ x_1 \\ \vdots \\ x_D \end{bmatrix} = [1, x_1, \cdots, x_D]^T$$

THEN:

$y = sign(\boldsymbol{w}^T \boldsymbol{x})$

IF:

$$\boldsymbol{w} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_D \end{bmatrix} = [w_1, w_2, \cdots, w_D]^T$$

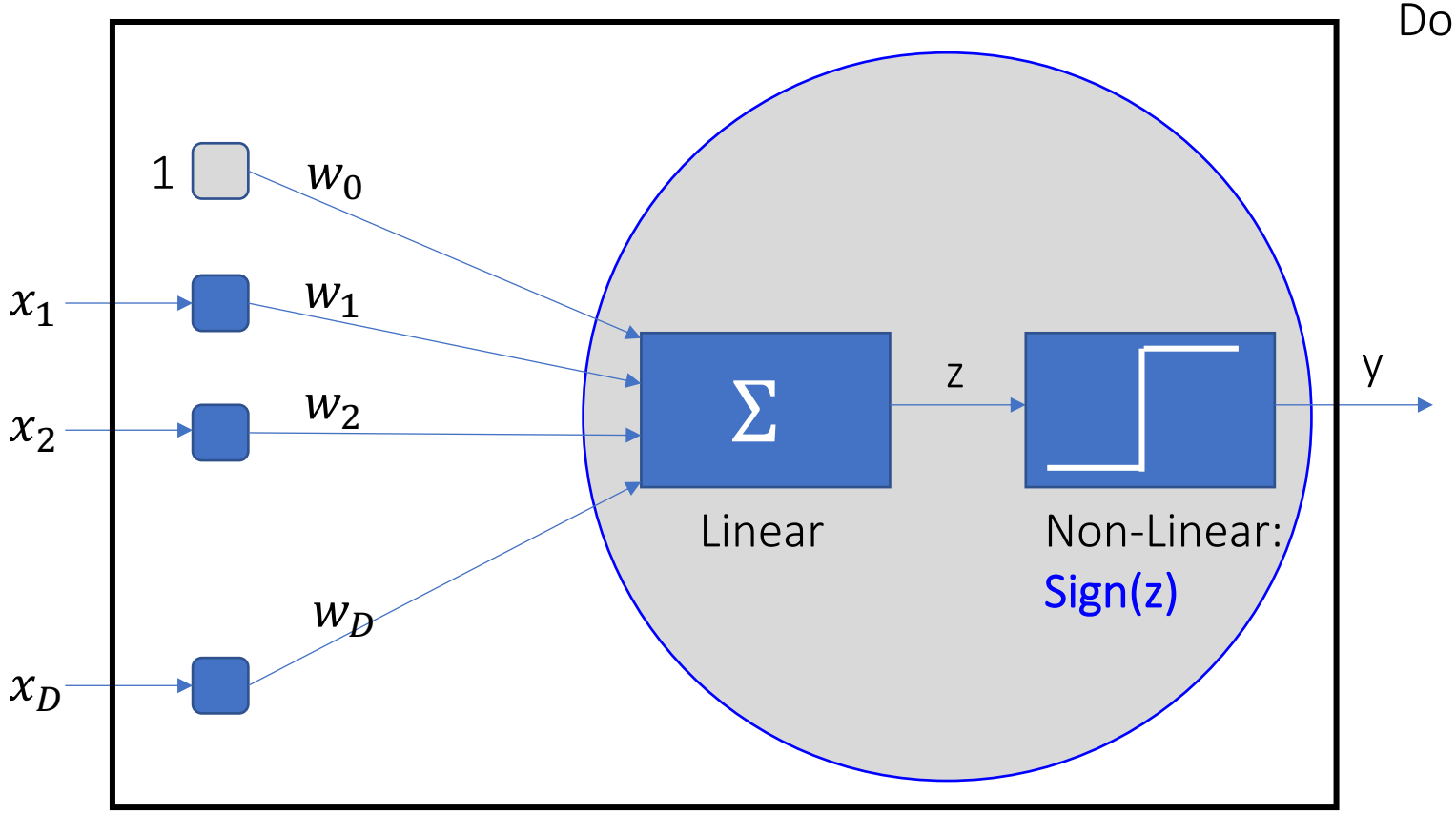
$$\boldsymbol{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_D \end{bmatrix} = [x_1, x_2, \cdots, x_D]^T$$

THEN:

$y = sign(\boldsymbol{w}^T \boldsymbol{x} + w_0)$

Or, $y = sign(\boldsymbol{w}^T \boldsymbol{x} + b); b = w_0$

Computational Graph



Dom(y) = {-1, 0, +1}. What is +1 and -1?

In some applications:

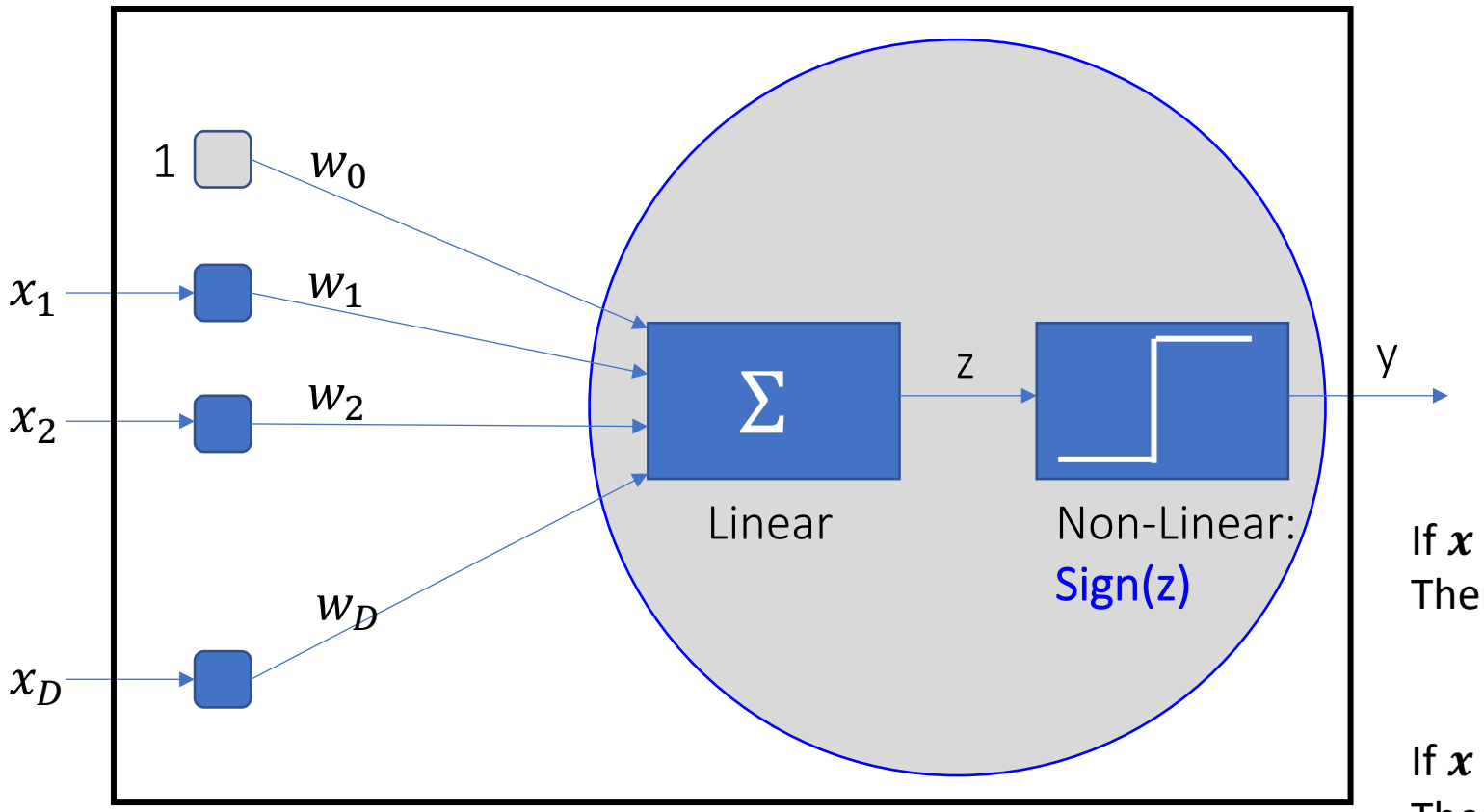
y = +1: Cat
y = -1: Dog

In others:

y = +1: Cancer
y = -1: Non-Caner

y = +1: Rainy
y = -1: Sunny

Model's Prediction



Example:

- D = 2
- Model's parameters:
  - $\boldsymbol{w} = [1, -4]^T;\ \mathrm{b} = 1$
- Classes:
  - {-1: NonCancer, +1: Cancer}

If $\boldsymbol{x} = [2, 1]^T$
Then:

- $y = sign(\boldsymbol{w}^T\boldsymbol{x} + \mathrm{b}) = -1 \equiv$ **Non-Cancer**

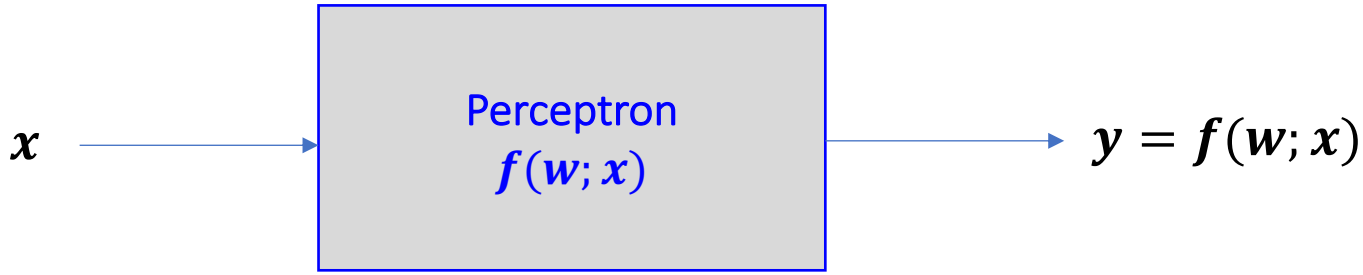
If $\boldsymbol{x} = [1, -1]^T$
Then:

- $y = sign(\boldsymbol{w}^T\boldsymbol{x} + \mathrm{b}) = +1 \equiv$ **Cancer**

Learning

Question: How can we have model's parameters

Answer: Learning the parameters from trainning data!



Learning

Question: How can we have model's parameters

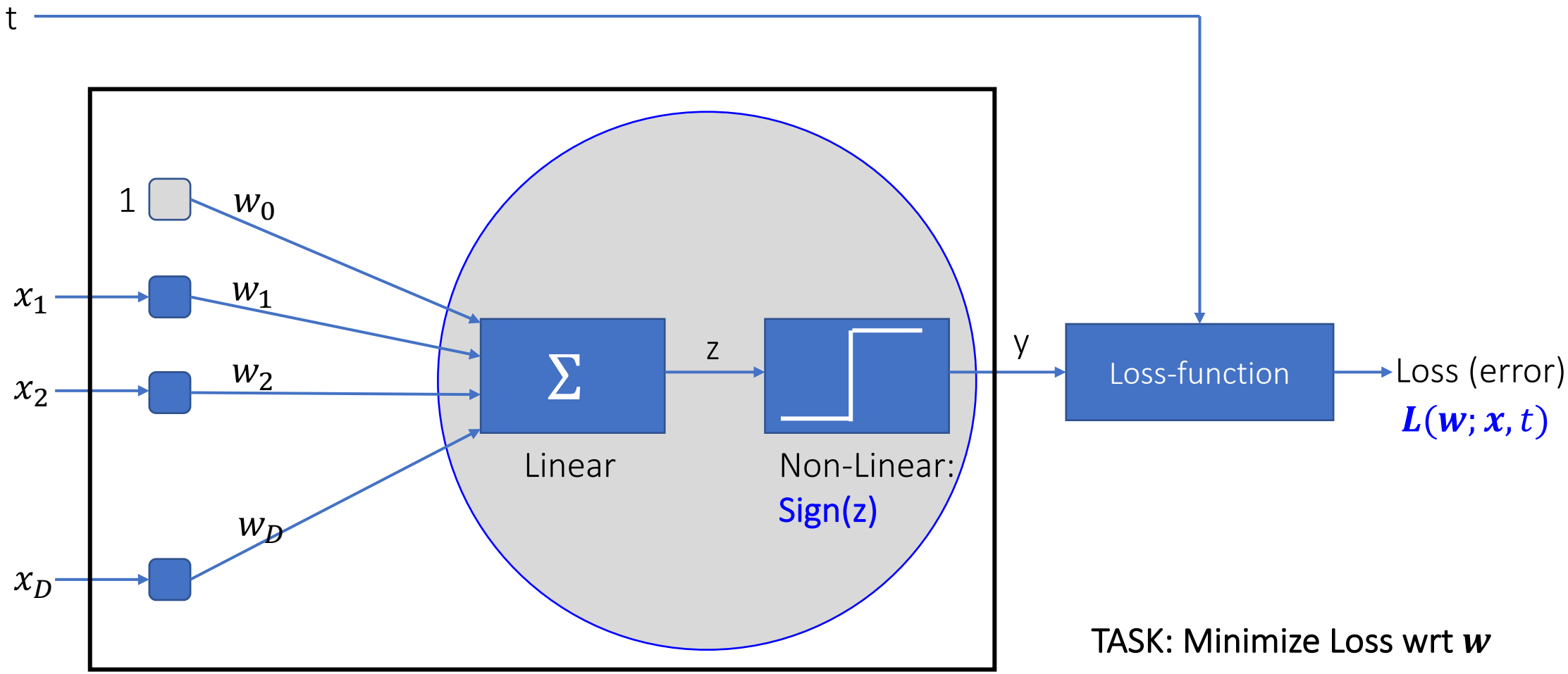
Answer: Learning the parameters from trainning dataset!

Training dataset: $< \boldsymbol{x}_1, \mathrm{t}_1 >, < \boldsymbol{x}_2, \mathrm{t}_2 >, \cdots, < \boldsymbol{x}_\mathrm{N}, \mathrm{t}_\mathrm{N} >$

Note:

- $\boldsymbol{x}_\mathrm{i}$ in D-dimensions space
- $\mathrm{t}_\mathrm{i} \in \{+1, -1\}$

Learning
Training diagram



Learning
Loss function

x_1	x_2	T (target)	y (model's prediction)	Match?	
a	b	+1	+1	Correct	
c	d	-1	+1	Error	=> Compute loss
e	f	-1	+1	Error	=> Compute loss
g	h	-1	-1	Correct	
i	k	+1	-1	Error	=> Compute loss

=> Set of miss-classified samples (for computing loss):

E = { <(c,d), -1>, <(e,f), -1>, <(i, k), +1> }

Learning
Loss function

x_1	x_2	t (target)	$w^t x + b$	$t * (w^t x + b)$	y (model's prediction)	Match?	
c	d	-1	> 0	< 0	+1	Error	=> Compute loss
e	f	-1	> 0	< 0	+1	Error	=> Compute loss
i	k	+1	< 0	< 0	-1	Error	=> Compute loss

$t * (\boldsymbol{w}^t \boldsymbol{x} + b)$: Always negative (<0) for miss-classified samples

$-t * (\boldsymbol{w}^t \boldsymbol{x} + b)$: Always positive (>0) for miss-classified samples

Learning
Loss function

$$L(\boldsymbol{w}) = \sum_{<\boldsymbol{x},t>\in E} -t * (\boldsymbol{w}^T\boldsymbol{x} + \mathrm{b}) \Rightarrow L(\boldsymbol{w};\boldsymbol{x},t) = -t * (\boldsymbol{w}^T\boldsymbol{x} + \mathrm{b})$$

$$\Rightarrow \nabla_w L(\boldsymbol{w};\boldsymbol{x},t) = -t * \boldsymbol{x} \quad \text{(derivative of L wrt } \mathbf{w}\text{)}$$

$$\nabla_{\mathrm{b}} L(\boldsymbol{w};\boldsymbol{x},t) = -t \quad \text{(derivative of L wrt b)}$$

Update: $w = w - \alpha * \nabla_w L$

=> $w = w + \alpha * t * \boldsymbol{x}$

Update: $\mathrm{b} = \mathrm{b} - \alpha * \nabla_{\mathrm{b}} L$

=> $\mathrm{b} = \mathrm{b} + \alpha * t$

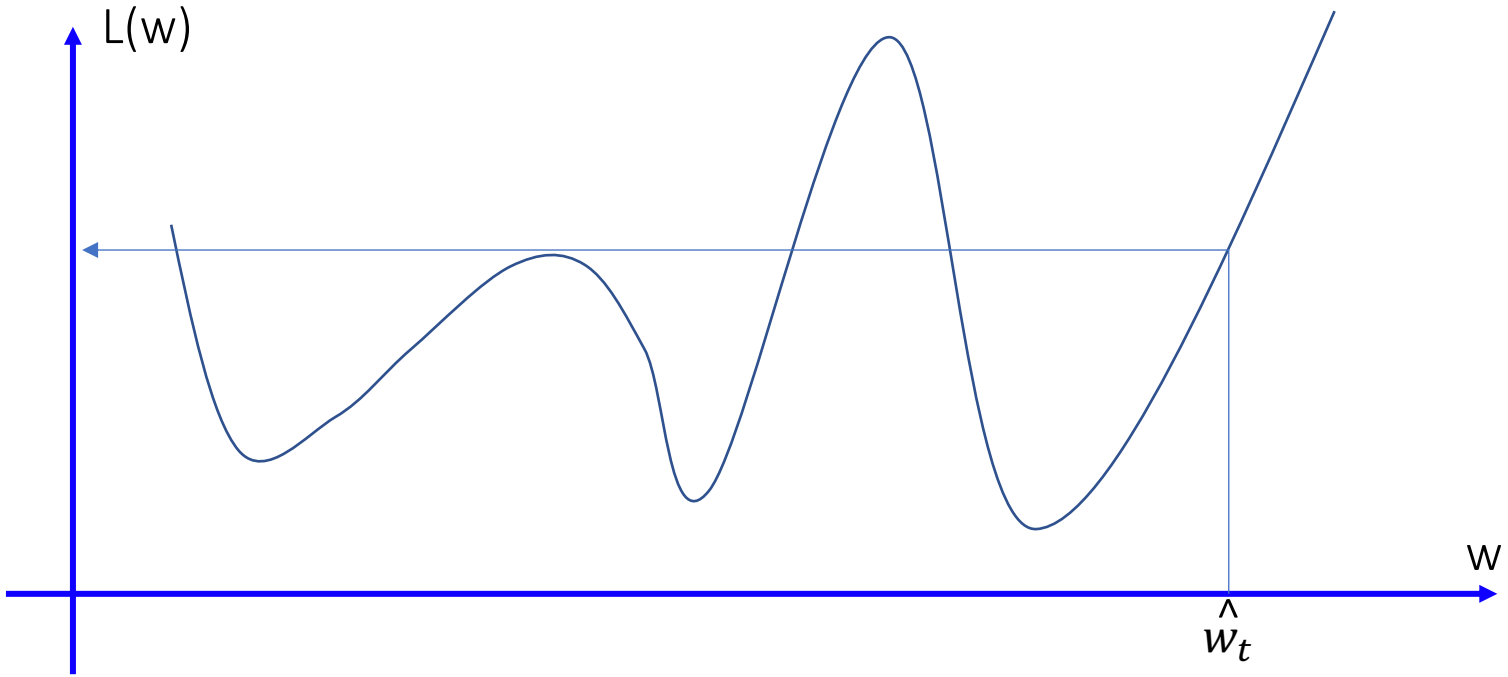
With α=1 :

$$\boldsymbol{w} = \boldsymbol{w} + t * \boldsymbol{x}$$

$$\mathrm{b} = \mathrm{b} + t$$

α: Learning rate (hyper-parameter, assigned before learning)

Stochastic Gradient Descent (SGD)

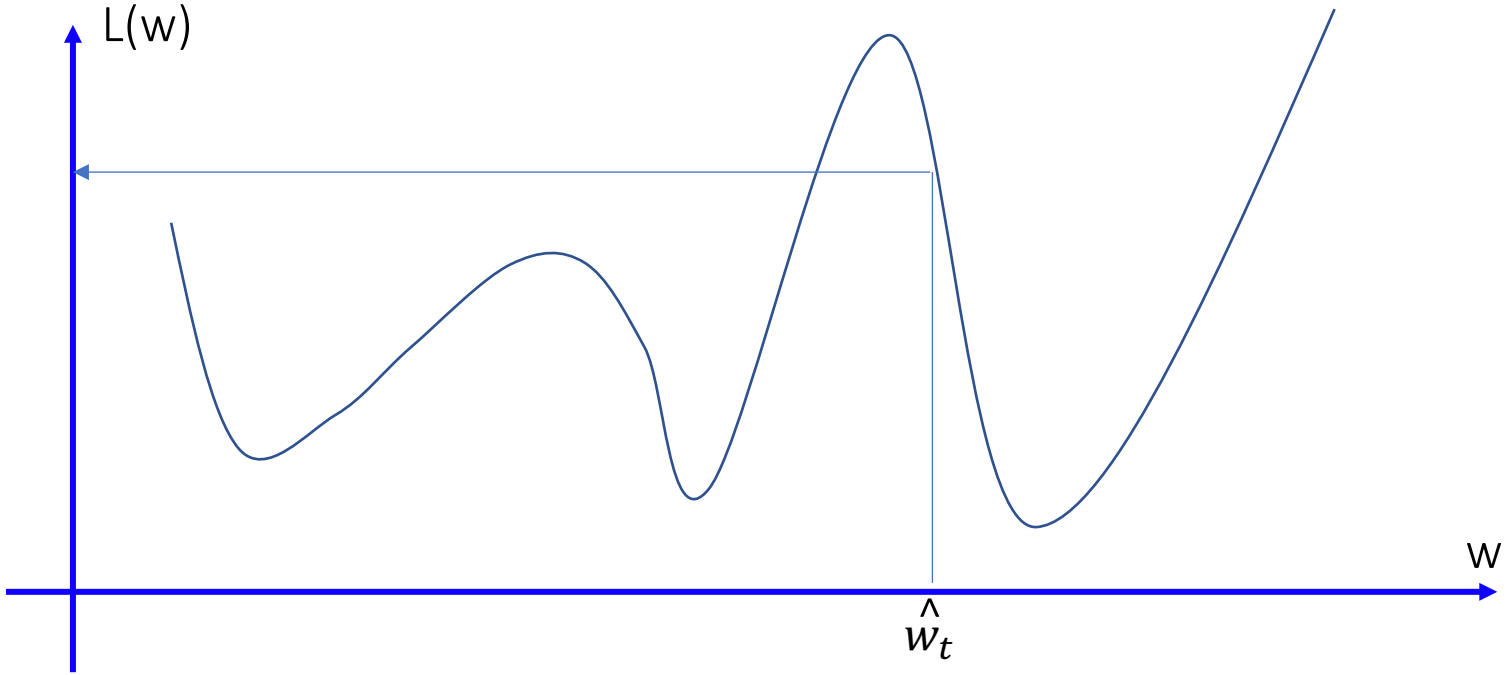


Current: w_t with loss: $\mathrm{L}(w_t)$

Need to **decrease** w_t to get smaller loss

MATH: Derivative of $\mathrm{L}(w_t)$ at w_t: positive

Stochastic Gradient Descent (SGD)

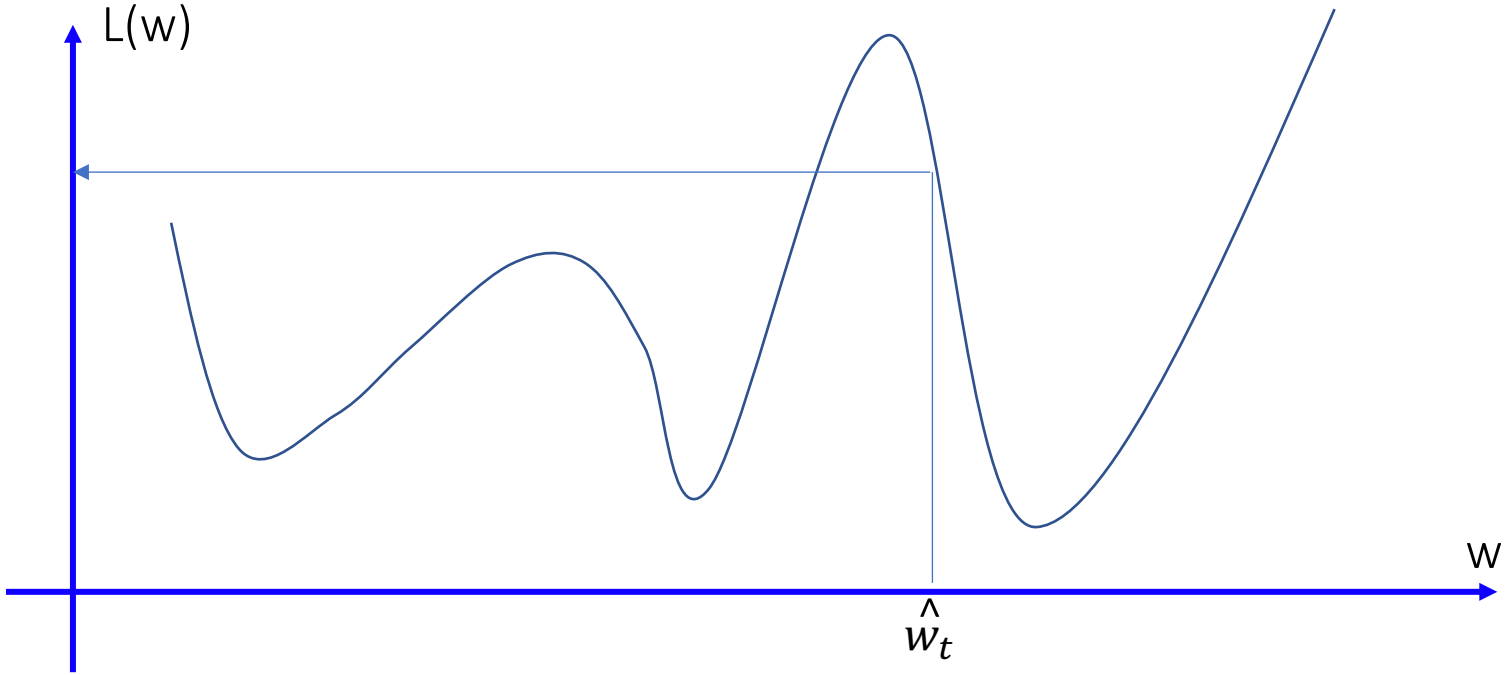


Current: w_t with loss: $\mathrm{L}(w_t)$

Need to **increase** w_t to get smaller loss

MATH: Derivative of $\mathrm{L}(w_t)$ at w_t: negative

Stochastic Gradient Descent (SGD)



For any point w_t:

IF update with: $w_{t+1} = w_t - \alpha * \nabla L(w_t)$;

where: $\nabla L(w)$: Derivative of $L(w_t)$ at w_t and small positive α

THEN $L(w_{t+1}) < L(w_t)$

Perceptron Learning Algorithm (PLA)

INPUT:

- Data: $\boldsymbol{X}$ (matrix NxD)
- Target: $\boldsymbol{t}$ (vector of N items)

(1) Initialize $\boldsymbol{w}, b$ with small values

(2) WHILE forever:

(2.1) Shuffle samples in the training set

(2.2) FOREACH sample $< \boldsymbol{x}, t >$ in the training set

(2.1.1) $\mathrm{y} = \mathbf{sign}(\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x} + \mathrm{b})$

(2.1.2) IF y <> t, update:

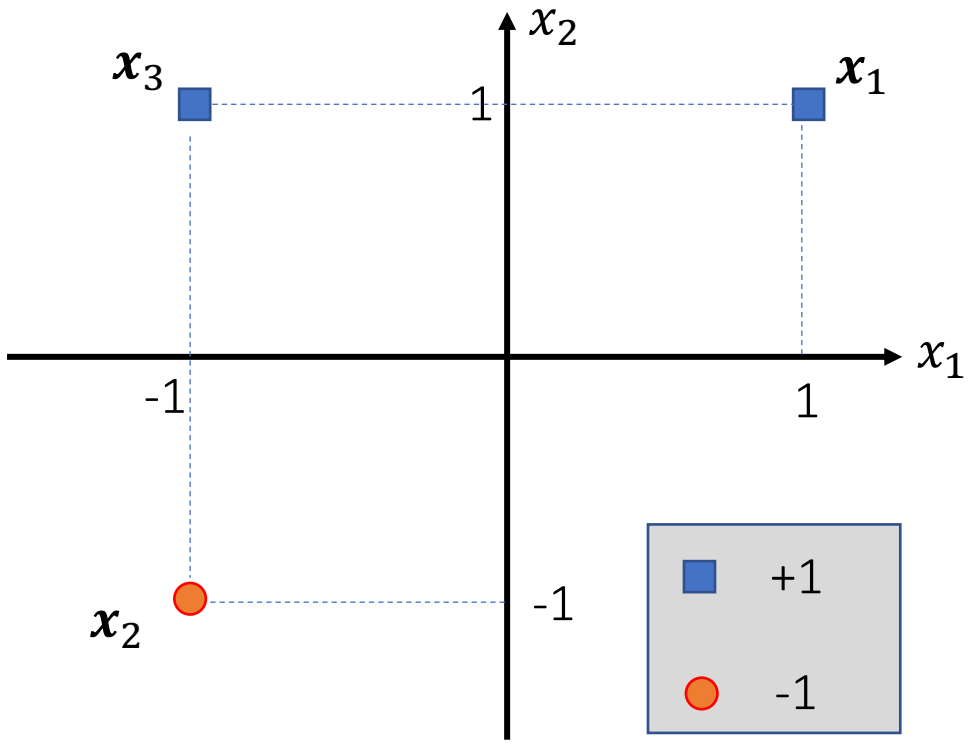
$$\boldsymbol{w} = \boldsymbol{w} + \mathrm{t} * \boldsymbol{x}$$

$$\mathrm{b} = \mathrm{b} + \mathrm{t}$$

(2.3) IF all samples classified correctly

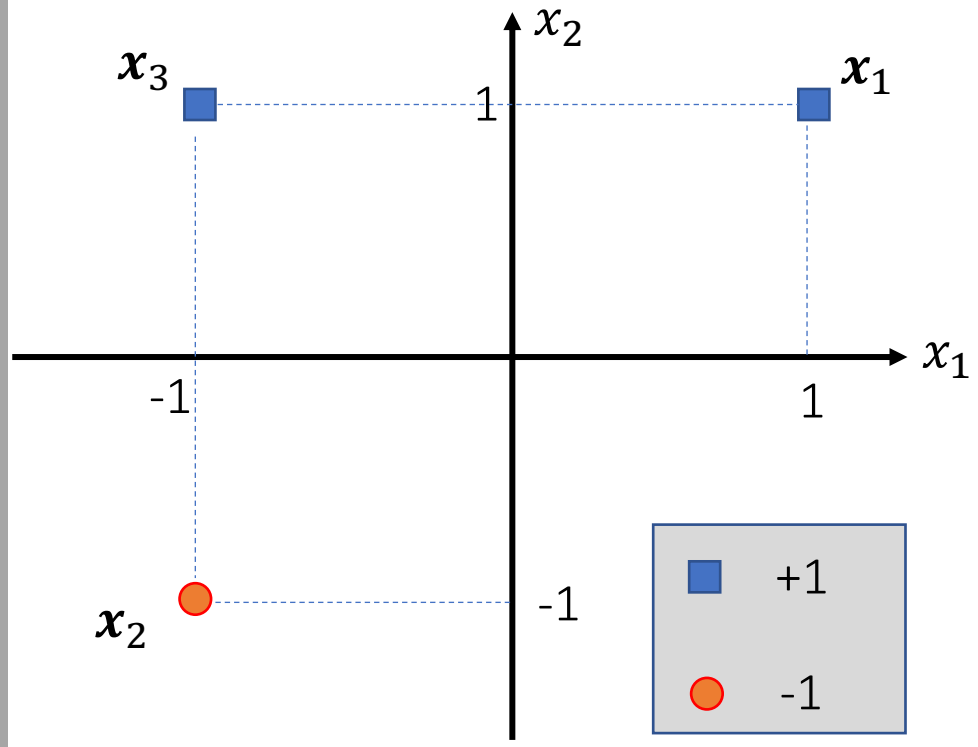
break WHILE (stop)

PLA – Step-by-Step



Important note:
x_1 (unbold, scalar) is different to $\boldsymbol{x}_1$ (**bold, vector**)

PLA – Step-by-Step



$$\boldsymbol{x}_1 = [1, 1]; t_1 = +1$$
$$\boldsymbol{x}_2 = [-1, -1]; t_2 = -1$$
$$\boldsymbol{x}_3 = [-1, 1]; t_2 = +1$$

Compressed representation:

$$\mathrm{X} = \begin{bmatrix} 1 & 1 \\ -1 & -1 \\ -1 & 1 \end{bmatrix} \quad t = \begin{bmatrix} +1 \\ -1 \\ +1 \end{bmatrix}$$

PLA – Step-by-Step

(1) Initialize (random): $\boldsymbol{w} = [1, -1]^T, \mathrm{b} = 0$

(2) Iteration-1 (WHILE)

(2.1) Shuffle:

$$\mathrm{X} = \begin{bmatrix} 1 & 1 \\ -1 & -1 \\ -1 & 1 \end{bmatrix} \quad t = \begin{bmatrix} +1 \\ -1 \\ +1 \end{bmatrix}$$ (assume that we get this result after shuffling)

(2.2) foreach $< x, t >$ in $< \boldsymbol{X}, \boldsymbol{t} >$:

* $\boldsymbol{x}_1 = [1,1]^T, \mathrm{t} = +1$

$\Rightarrow \mathrm{y} = \mathrm{sign}(\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}_1 + \mathrm{b}) = \mathrm{sign}([1, -1]^{\mathrm{T}}.[1, 1]^{\mathrm{T}} + 0) = 0$

y <> t => update:

$\boldsymbol{w} = \boldsymbol{w} + \mathrm{t} * \boldsymbol{x}_1 = [1, -1]^T + [1,1]^T = [2,0]^T$

$\mathrm{b} = \mathrm{b} + \mathrm{t} = 1$

PLA – Step-by-Step

(1) Initialize (random): $\boldsymbol{w} = [1, -1]^T, \mathrm{b} = 0$

(2) Iteration-1 (WHILE)

(2.1) Shuffle:

$$\mathrm{X} = \begin{bmatrix} 1 & 1 \\ -1 & -1 \\ -1 & 1 \end{bmatrix} \quad t = \begin{bmatrix} +1 \\ -1 \\ +1 \end{bmatrix}$$

(2.2) foreach $< x, t >$ in $< \boldsymbol{X}, \boldsymbol{t} >$:

* $\boldsymbol{x}_2 = [-1, -1]^T, \mathrm{t} = -1$

$$\Rightarrow \mathrm{y} = \mathrm{sign}\left(\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}_2 + \mathrm{b}\right) = \mathrm{sign}\left([2, 0]^{\mathrm{T}}.[-1, -1]^{\mathrm{T}} + 1\right) = -1$$

$y == t$ => no update:

PLA – Step-by-Step

(1) Initialize (random): $\boldsymbol{w} = [1, -1]^T, \mathrm{b} = 0$

(2) Iteration-1 (WHILE)

(2.1) Shuffle:

$$\mathrm{X} = \begin{bmatrix} 1 & 1 \\ -1 & -1 \\ -1 & 1 \end{bmatrix} \quad t = \begin{bmatrix} +1 \\ -1 \\ +1 \end{bmatrix}$$

(2.2) foreach $< x, t >$ in $< \boldsymbol{X}, \boldsymbol{t} >$:

$* \boldsymbol{x}_3 = [-1, 1]^T, \mathrm{t} = +1$

$$\Rightarrow \mathrm{y} = \mathrm{sign}\left(\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}_3 + \mathrm{b}\right) = \mathrm{sign}\left([2,0]^{\mathrm{T}}.[-1,1]^{\mathrm{T}} + 1\right) = -1$$

$y <> t$ => update:

$$\boldsymbol{w} = \boldsymbol{w} + \mathrm{t} * \boldsymbol{x}_3 = [2,0]^T + [-1,1]^T = [1,1]^T$$

$$\mathrm{b} = \mathrm{b} + \mathrm{t} = 1 + 1 = 2$$

PLA – Step-by-Step

(1) Initialize (random): $\boldsymbol{w} = [1, -1]^T, \mathrm{b} = 0$

(2) Iteration-1 (WHILE)

(2.1) Shuffle:

$$\mathrm{X} = \begin{bmatrix} 1 & 1 \\ -1 & -1 \\ -1 & 1 \end{bmatrix} \quad t = \begin{bmatrix} +1 \\ -1 \\ +1 \end{bmatrix}$$

(2.2) foreach $< x, t >$ in $< \boldsymbol{X}, \boldsymbol{t} >$:

(2.3) IF all samples classified correctly

* $\boldsymbol{x}_1 = [1,1]^T, \mathrm{t} = +1$

$$\Rightarrow \mathrm{y} = \mathrm{sign}(\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}_1 + \mathrm{b}) = \mathrm{sign}([1,1]^{\mathrm{T}}.[1,1]^{\mathrm{T}} + 2) = +1 \text{ (correctly)}$$

* $\boldsymbol{x}_2 = [-1,-1]^T, \mathrm{t} = -1$

$$\Rightarrow \mathrm{y} = \mathrm{sign}(\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}_2 + \mathrm{b}) = \mathrm{sign}([1,1]^{\mathrm{T}}.[-1,-1]^{\mathrm{T}} + 2) = 0 \text{ (incorrectly)}$$

PLA – Step-by-Step

(1) Initialize (random): $\boldsymbol{w} = [1, -1]^T, \mathrm{b} = 0$

(2) Iteration-2 (WHILE)

(2.1) Shuffle:

$$\mathrm{X} = \begin{bmatrix} -1 & -1 \\ 1 & 1 \\ -1 & 1 \end{bmatrix} \quad t = \begin{bmatrix} -1 \\ +1 \\ +1 \end{bmatrix}$$

(2.2) foreach $< x, t >$ in $< \boldsymbol{X}, \boldsymbol{t} >$:

* $\boldsymbol{x}_1 = [-1, -1]^T, \mathrm{t} = -1$

$\Rightarrow \mathrm{y} = \mathrm{sign}(\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}_1 + \mathrm{b}) = \mathrm{sign}([1,1]^{\mathrm{T}}.[-1,-1]^{\mathrm{T}} + 2) = 0$

y <> t => update:

$\boldsymbol{w} = \boldsymbol{w} + \mathrm{t} * \boldsymbol{x}_1 = [1,1]^T - [-1,-1]^T = [2,2]^T$

$\mathrm{b} = \mathrm{b} + \mathrm{t} = 2 - 1 = 1$

PLA – Step-by-Step

(1) Initialize (random): $\boldsymbol{w} = [1, -1]^T, \mathrm{b} = 0$

(2) Iteration-2 (WHILE)

(2.1) Shuffle:

$$\mathrm{X} = \begin{bmatrix} -1 & -1 \\ 1 & 1 \\ -1 & 1 \end{bmatrix} \quad t = \begin{bmatrix} -1 \\ +1 \\ +1 \end{bmatrix}$$

(2.2) foreach $< x, t >$ in $< \boldsymbol{X}, \boldsymbol{t} >$:

* $\boldsymbol{x}_2 = [1,1]^T, \mathrm{t} = +1$

$\Rightarrow \mathrm{y} = \mathrm{sign}(\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}_2 + \mathrm{b}) = \mathrm{sign}([2,2]^{\mathrm{T}}.[-1,-1]^{\mathrm{T}} + 1) = -1$

y <> t => update:

$\boldsymbol{w} = \boldsymbol{w} + \mathrm{t} * \boldsymbol{x}_2 = [2,2]^T + [1,1]^T = [3,3]^T$

$\mathrm{b} = \mathrm{b} + \mathrm{t} = 1 + 1 = 2$

PLA – Step-by-Step

(1) Initialize (random): $\boldsymbol{w} = [1, -1]^T, \mathrm{b} = 0$

(2) Iteration-2 (WHILE)

(2.1) Shuffle:

$$\mathrm{X} = \begin{bmatrix} -1 & -1 \\ 1 & 1 \\ -1 & 1 \end{bmatrix} \quad t = \begin{bmatrix} -1 \\ +1 \\ +1 \end{bmatrix}$$

(2.2) foreach $< x, t >$ in $< \boldsymbol{X}, \boldsymbol{t} >$:

* $\boldsymbol{x}_3 = [-1,1]^T, \mathrm{t} = +1$

$$\Rightarrow \mathrm{y} = \mathrm{sign}\left(\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}_3 + \mathrm{b}\right) = \mathrm{sign}\left([3,3]^{\mathrm{T}}.[-1,1]^{\mathrm{T}} + 2\right) = +1$$

$y == t$ => no update:

PLA – Step-by-Step

(1) Initialize (random): $\boldsymbol{w} = [1, -1]^T, \mathrm{b} = 0$

(2) Iteration-2 (WHILE)

(2.1) Shuffle:

$$\mathrm{X} = \begin{bmatrix} -1 & -1 \\ 1 & 1 \\ -1 & 1 \end{bmatrix} \quad t = \begin{bmatrix} -1 \\ +1 \\ +1 \end{bmatrix}$$

(2.2) foreach $< x, t >$ in $< \boldsymbol{X}, \boldsymbol{t} >$:

(2.3) IF all samples classified correctly

* $\boldsymbol{x}_1 = [-1, -1]^T, \mathrm{t} = -1$

  $\Rightarrow \mathrm{y} = \mathrm{sign}(\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}_1 + \mathrm{b}) = \mathrm{sign}([3,3]^{\mathrm{T}}.[-1,-1]^{\mathrm{T}} + 2) = -1$ (correctly)

* $\boldsymbol{x}_2 = [1,1]^T, \mathrm{t} = +1$

  $\Rightarrow \mathrm{y} = \mathrm{sign}(\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}_2 + \mathrm{b}) = \mathrm{sign}([3,3]^{\mathrm{T}}.[1,1]^{\mathrm{T}} + 2) = +1$ (correctly)

* $\boldsymbol{x}_3 = [-1,1]^T, \mathrm{t} = +1$

  $\Rightarrow \mathrm{y} = \mathrm{sign}(\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}_3 + \mathrm{b}) = \mathrm{sign}([3,3]^{\mathrm{T}}.[-1,1]^{\mathrm{T}} + 2) = +1$ (correctly)

PLA – Step-by-Step

(1) Initialize (random): $\boldsymbol{w} = [1, -1]^T$, b $= 0$

(2) Iteration-2 (WHILE)

(2.1) Shuffle:

$$\mathrm{X} = \begin{bmatrix} -1 & -1 \\ 1 & 1 \\ -1 & 1 \end{bmatrix} \quad t = \begin{bmatrix} -1 \\ +1 \\ +1 \end{bmatrix}$$

(2.2) foreach $< x, t >$ in $< \boldsymbol{X}, \boldsymbol{t} >$:

(2.3) IF all samples classified correctly

All samples classified correctly => STOP

$\boldsymbol{w} = [3,3]^{\mathrm{T}}$

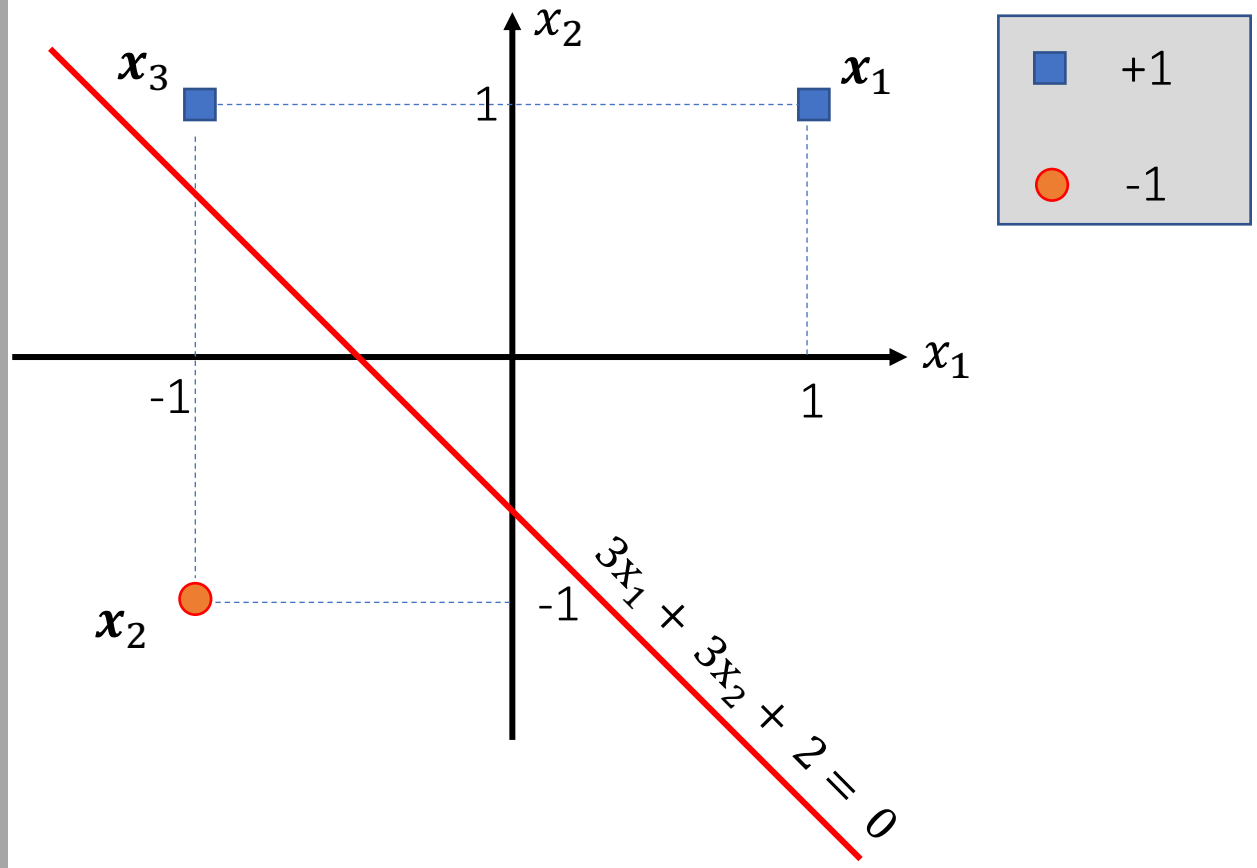
$b = 2$

Decision boundary equation:

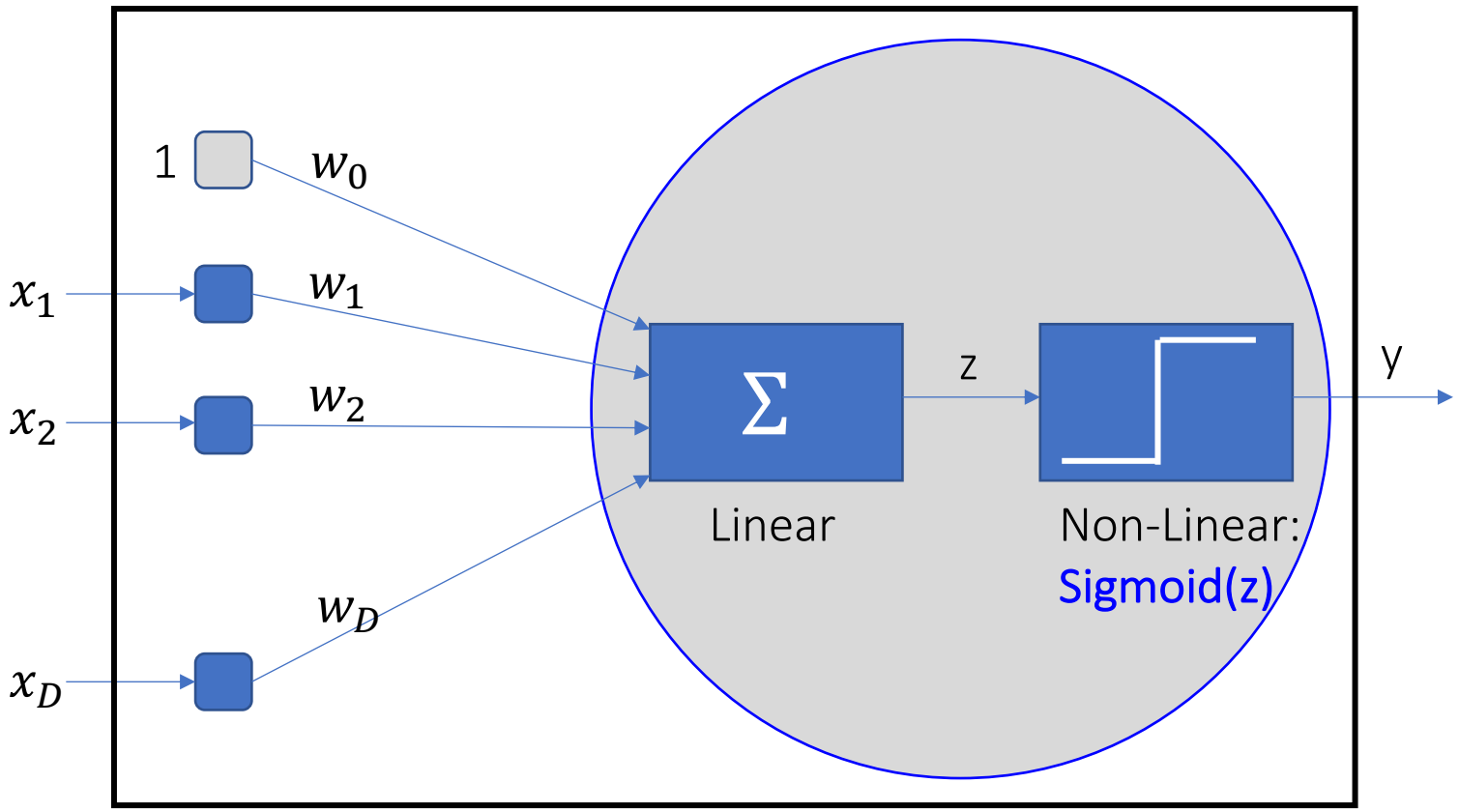
$\mathrm{w}^{\mathrm{T}}\mathrm{x} + \mathrm{b} = 0$

$3\mathrm{x}_1 + 3\mathrm{x}_2 + 2 = 0$

PLA – Step-by-Step



Other kinds of classification models

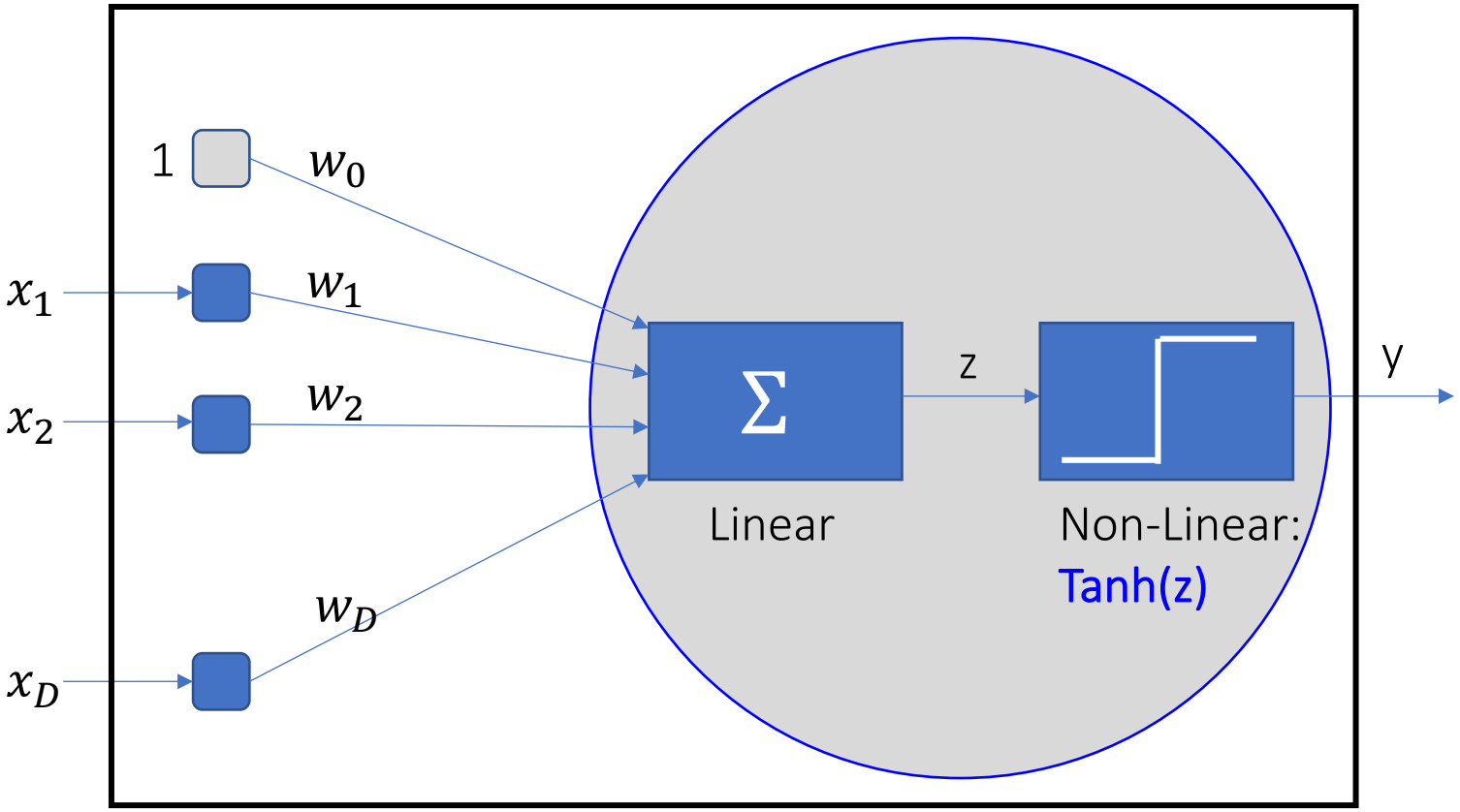


y >= 0.5: Cat
y < 0.5: Dog

y >= 0.5: Cancer
y < 0.5: Non-Cancer

y >= 0.5: Ranny
y < 0.5: Sunny

Other kinds of classification models

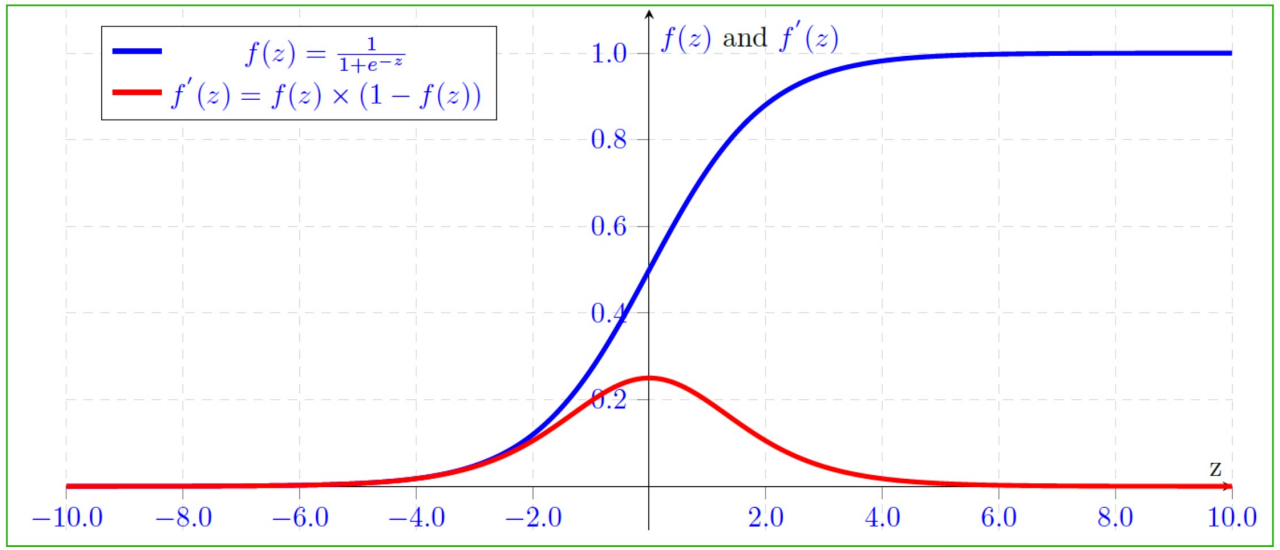


y >= 0: Cat
y < 0: Dog

y >= 0: Cancer
y < 0: Non-Cancer

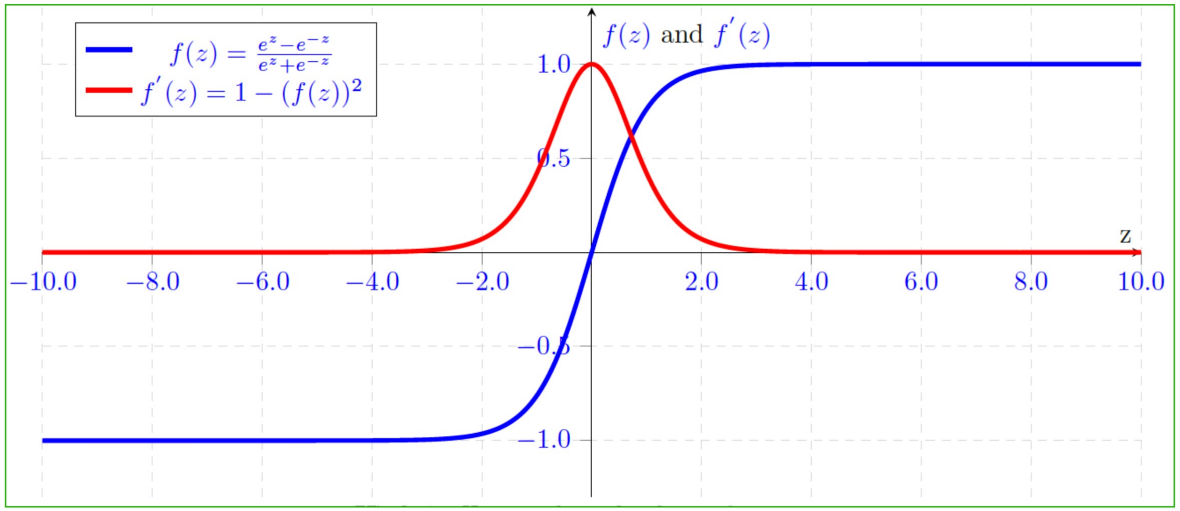
y >= 0: Ranny
y < 0: Sunny

Other kinds of classification models



<= Sigmoid

Tanh =>



Other kinds of classification models

Example, n-classes: [Cat, Dog, Chicken]